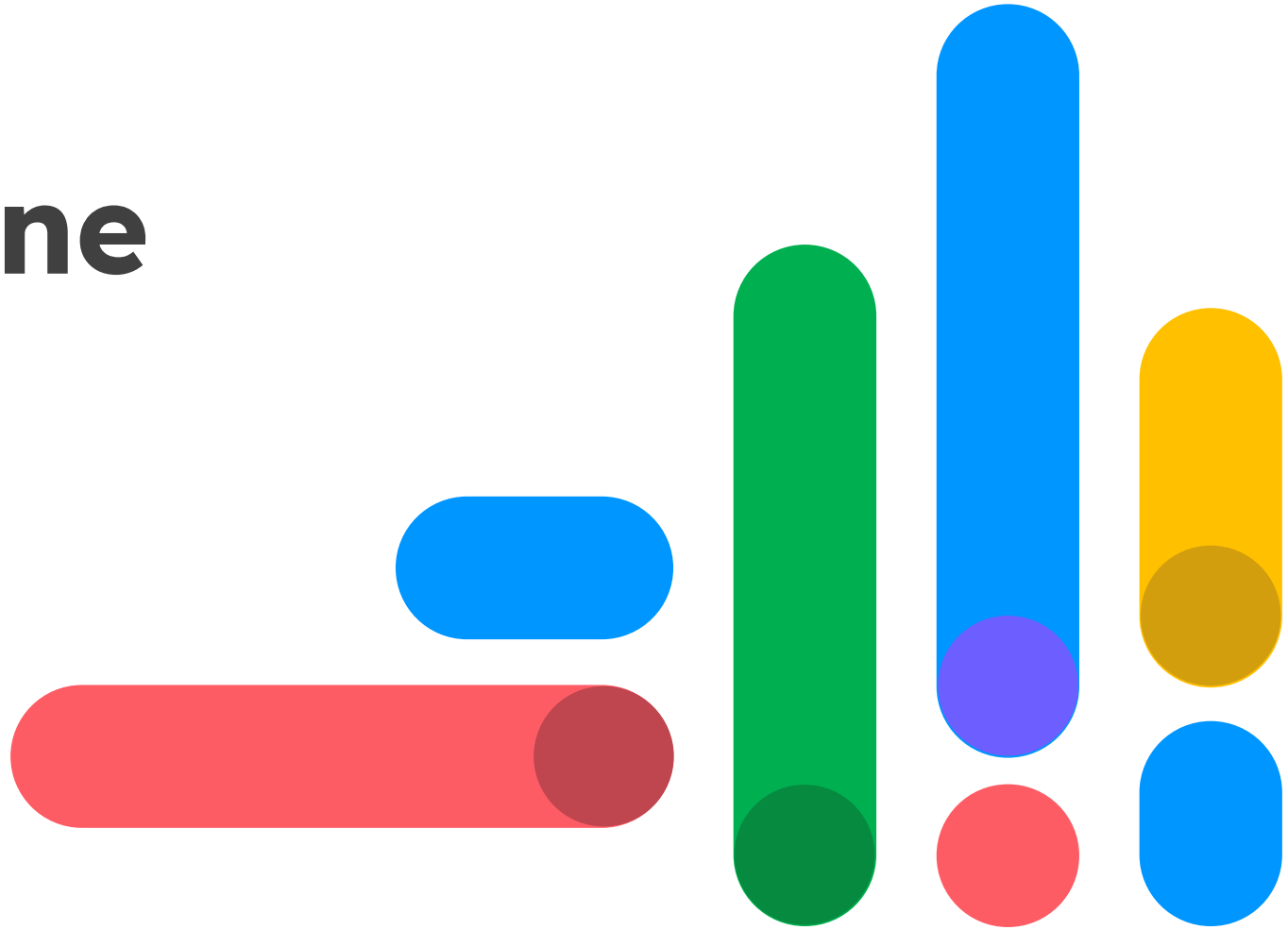
Deep Learning

Applied Machine Learning

The Fundamentals :: Basic

Assistant Professor | **Chun-Hsiang Chan**

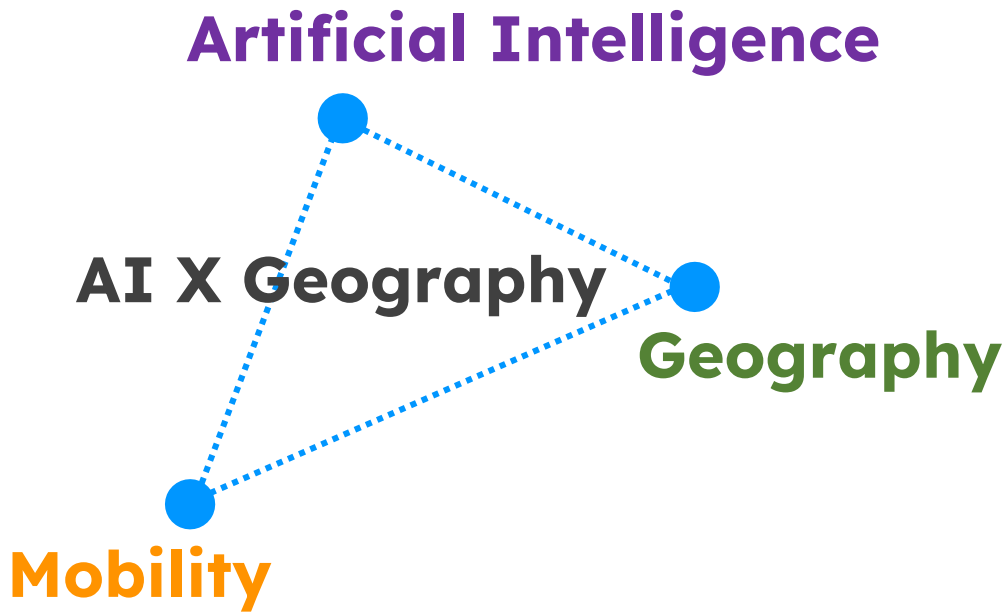
Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

01 Deep Learning
02 Model Formulation
03 Single Neuron
04 A Layer of Neurons
05 Hidden Layer
06 Activation Function
07 Linearity and Non-linearity
08 Model Parameters
09 Loss Function



Deep Learning

Deep learning (DL) neural networks, or artificial neural networks, attempts to mimic the human brain through a combination of **data inputs, weights, and bias**. These elements work together to accurately recognize, classify, and describe objects within the data.

Deep neural networks consist of multiple layers of interconnected nodes, each building upon the **previous layer to refine and optimize the prediction or categorization**.

Deep Learning

This progression of computations through the network is called forward propagation. The input and output layers of a deep neural network are called *visible* **layers**. The **input layer** is where the deep learning model ingests the data for processing, and the **output layer** is where the final prediction or classification is made.

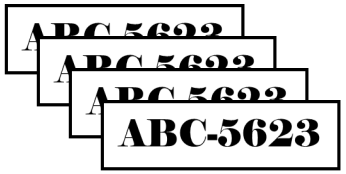
Model Formulation

Imagine that you have a problem, such as a digit recognition, object detection, or numerical prediction.

Digit recognition



Input



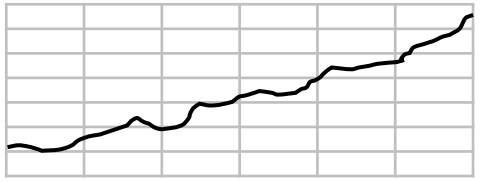
Function

$f(x)$

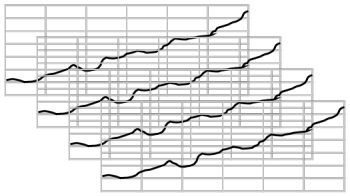
Output

> "ABC-5623"

Numerical prediction



Input



Function

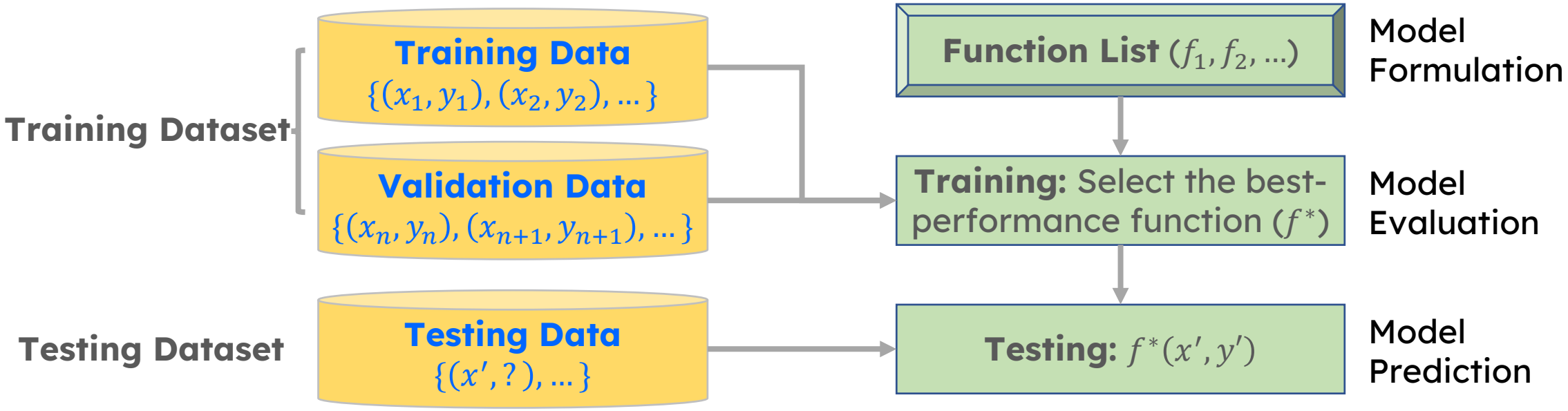
$f(x)$

Output

> 234, 235, …

Model Formulation

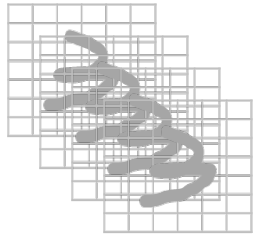
In machine learning and deep learning, we usually divide our input data into training and validation datasets for the training model.

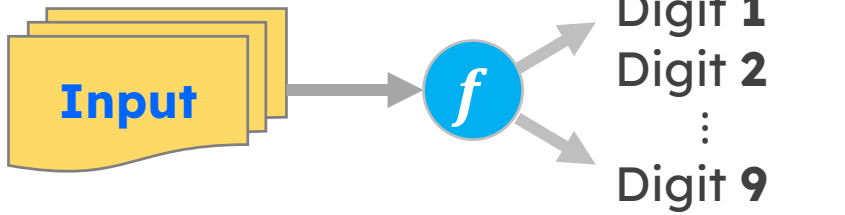


Model Formulation

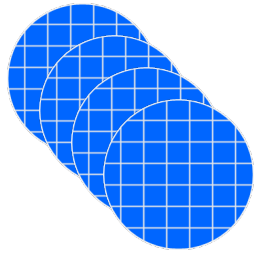
Prediction Tasks

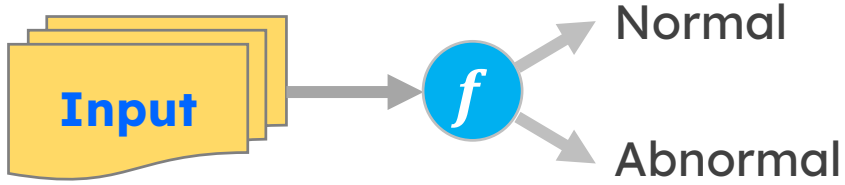
Handwritten Recognition



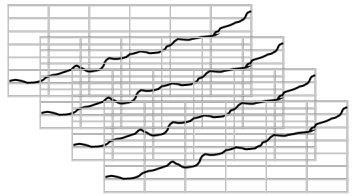


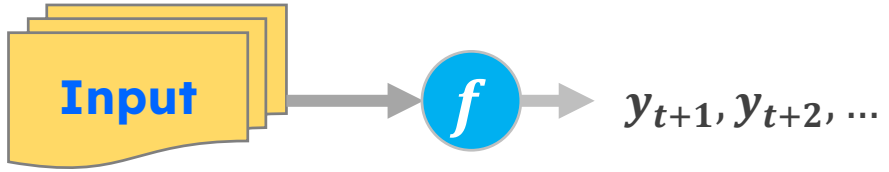
Abnormal Chip Testing





Stock Prediction

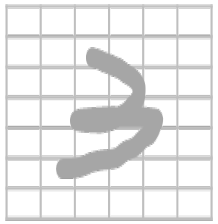




Model Formulation

Handwritten Recognition: alphabet and number

x: *image*



7 x 6 array

f

$f: R^N \rightarrow R^M$

y: *label*

1 indicates the element contains ink
0 indicates the element contains nothing

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \Rightarrow 7 \times 6 = 42 \; elements$$

10+26 dimension for label recognition

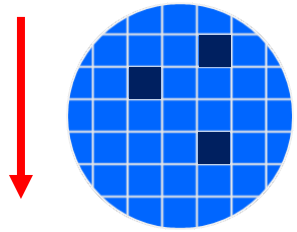
$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{matrix} a \\ b \\ c \\ \vdots \\ 8 \\ 9 \end{matrix} \quad \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Detect Ground Truth

Model Formulation

Abnormal Chip Testing

$x: image$



f

$y: label$

2 dimensions

Normal chip or Defect chip

1 indicates a defect chip
0 indicates a normal chip

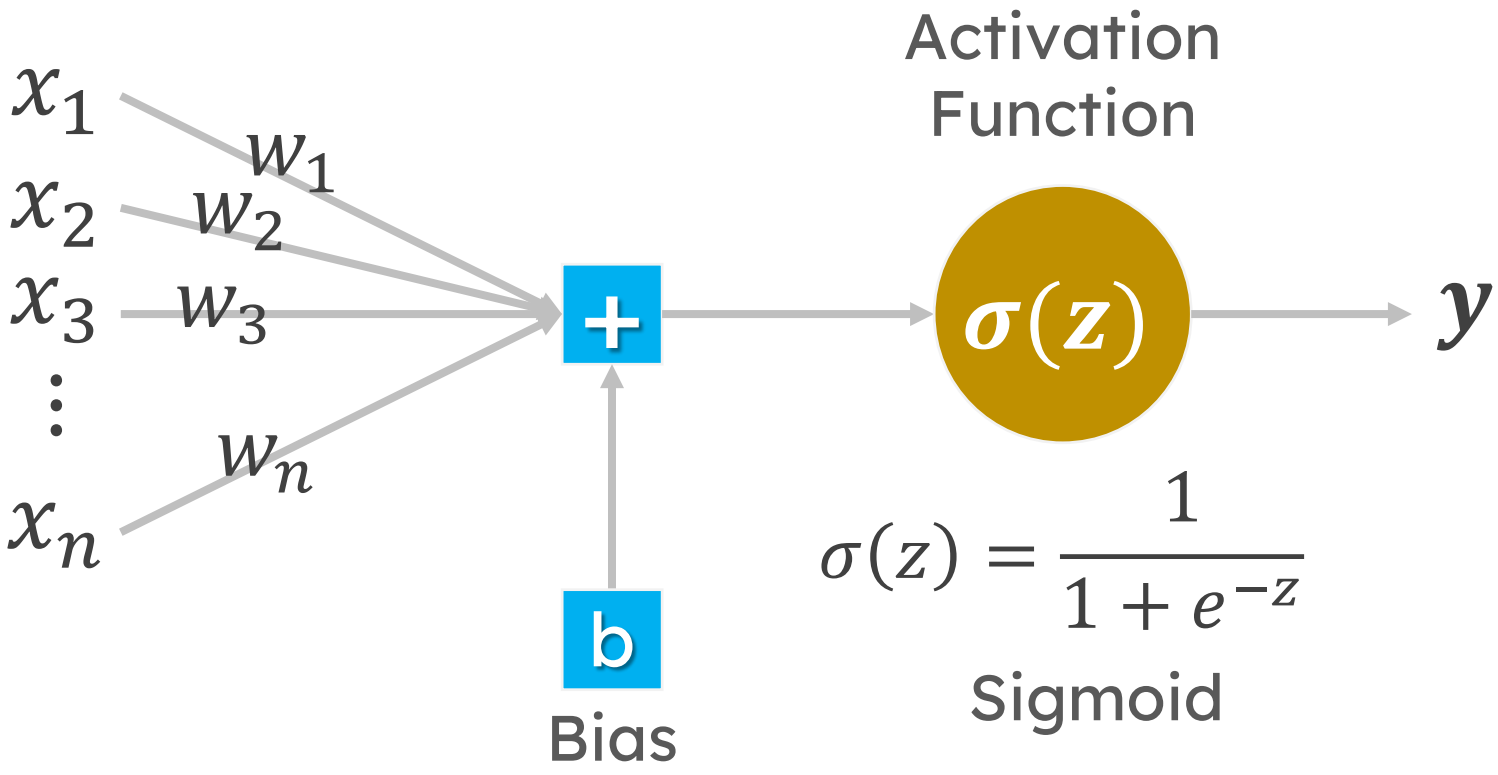
$$\begin{bmatrix}0\\0\\1\\0\\\vdots\\0\end{bmatrix} \Rightarrow dimensions = size(chips)$$

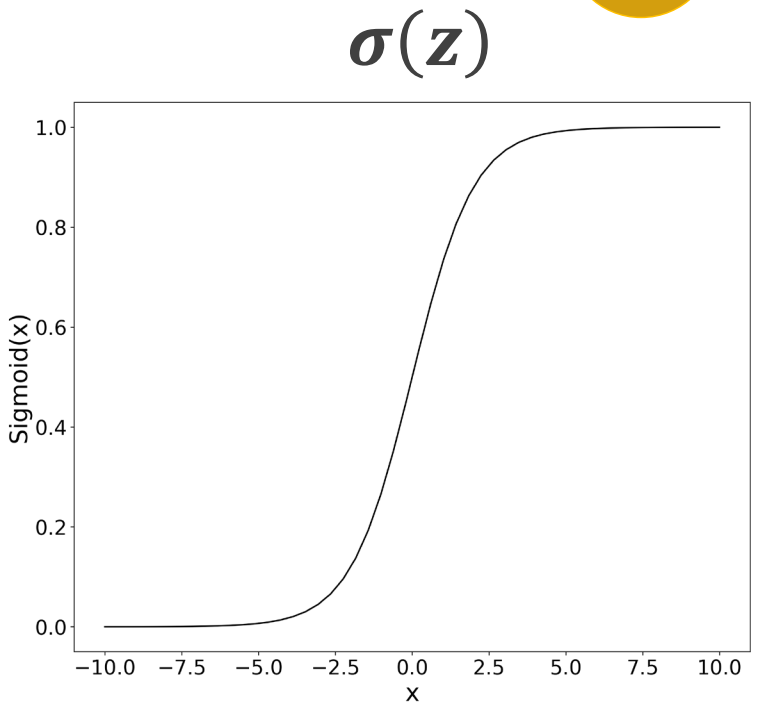
$$\begin{bmatrix}0\\0\\1\\0\\\vdots\\0\end{bmatrix} \begin{matrix}V\\V\\\times\\\vdots\\V\\V\end{matrix} \begin{bmatrix}0\\0\\1\\0\\\vdots\\0\end{bmatrix}$$

Confirm

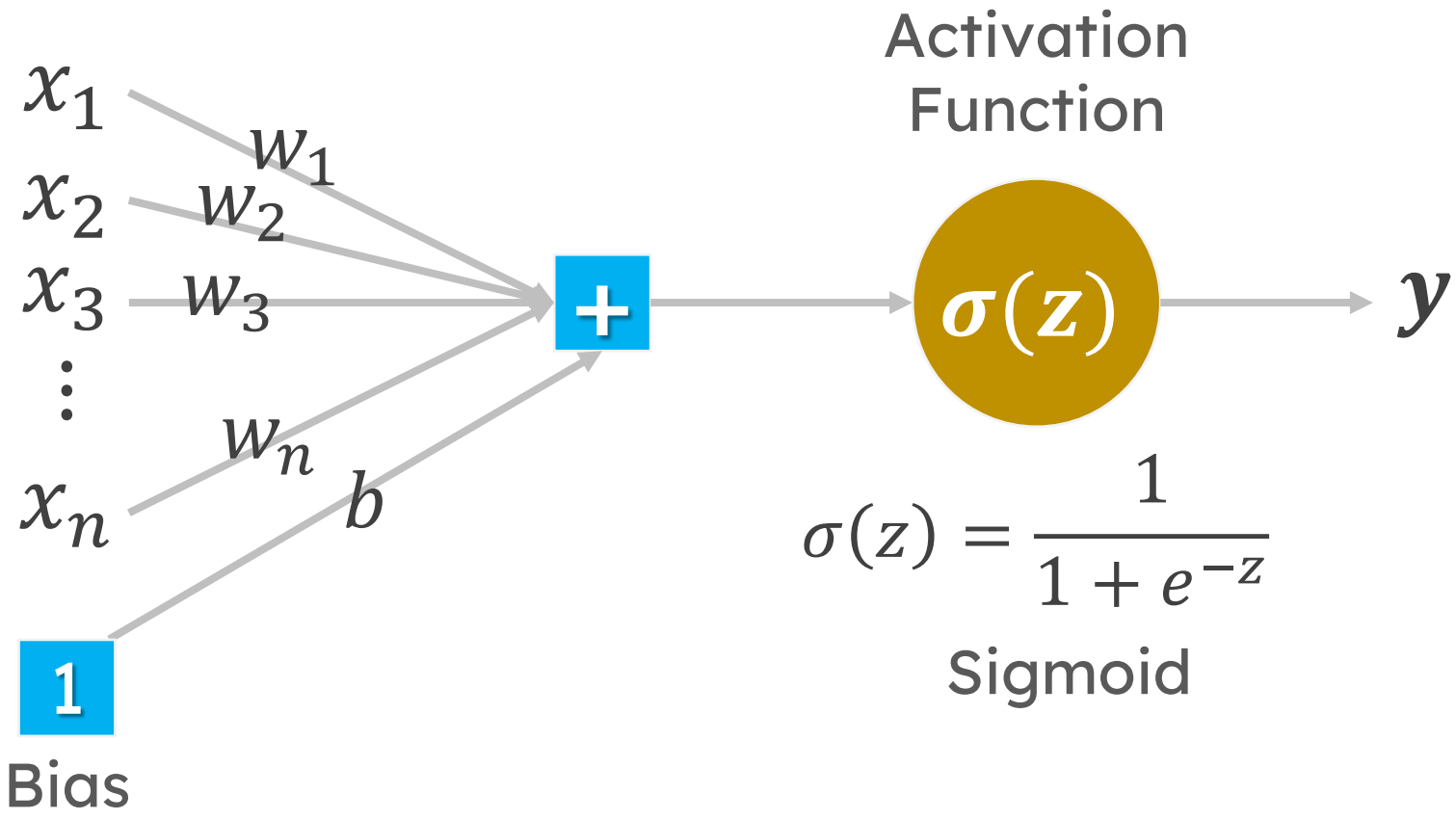
Detect Ground Truth

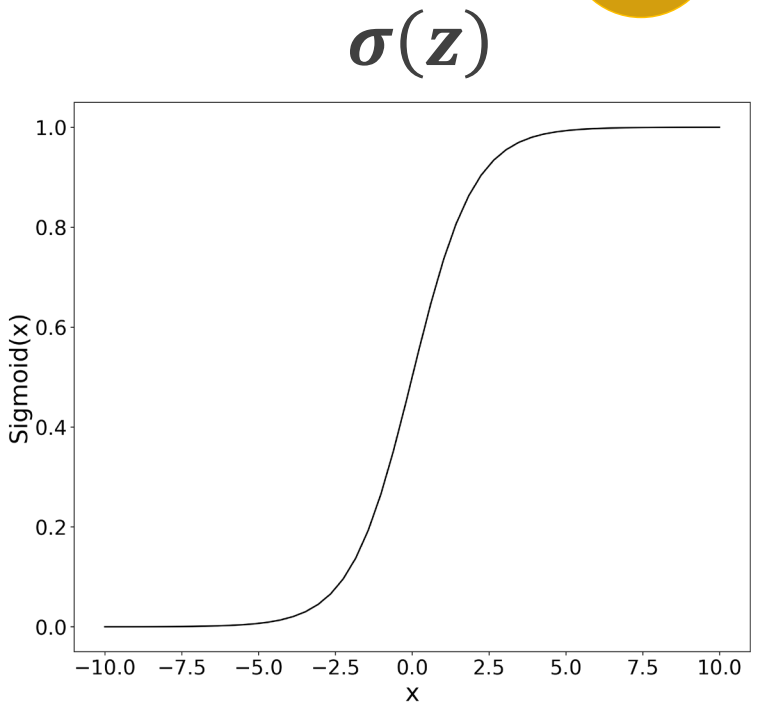
Single Neuron



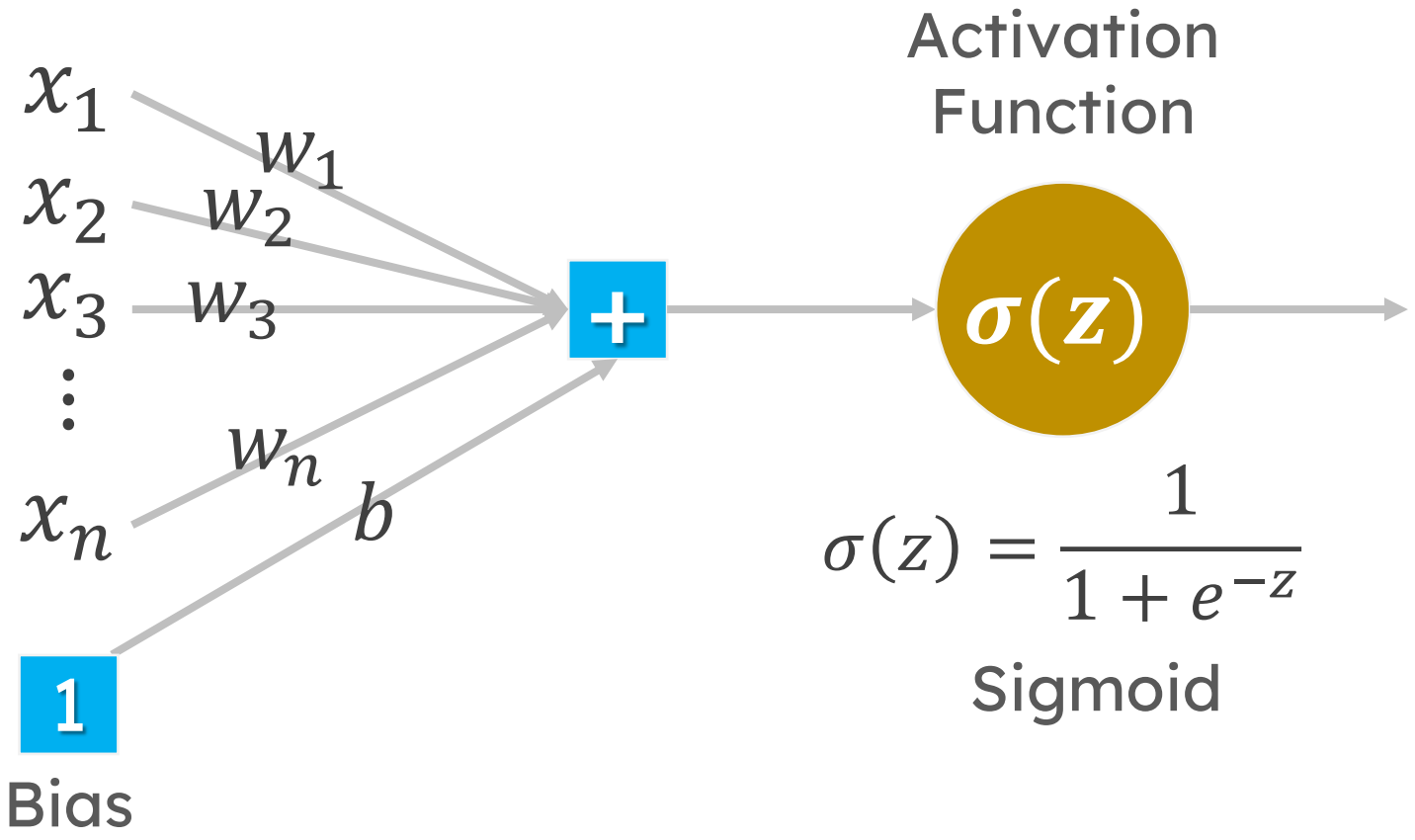


Single Neuron



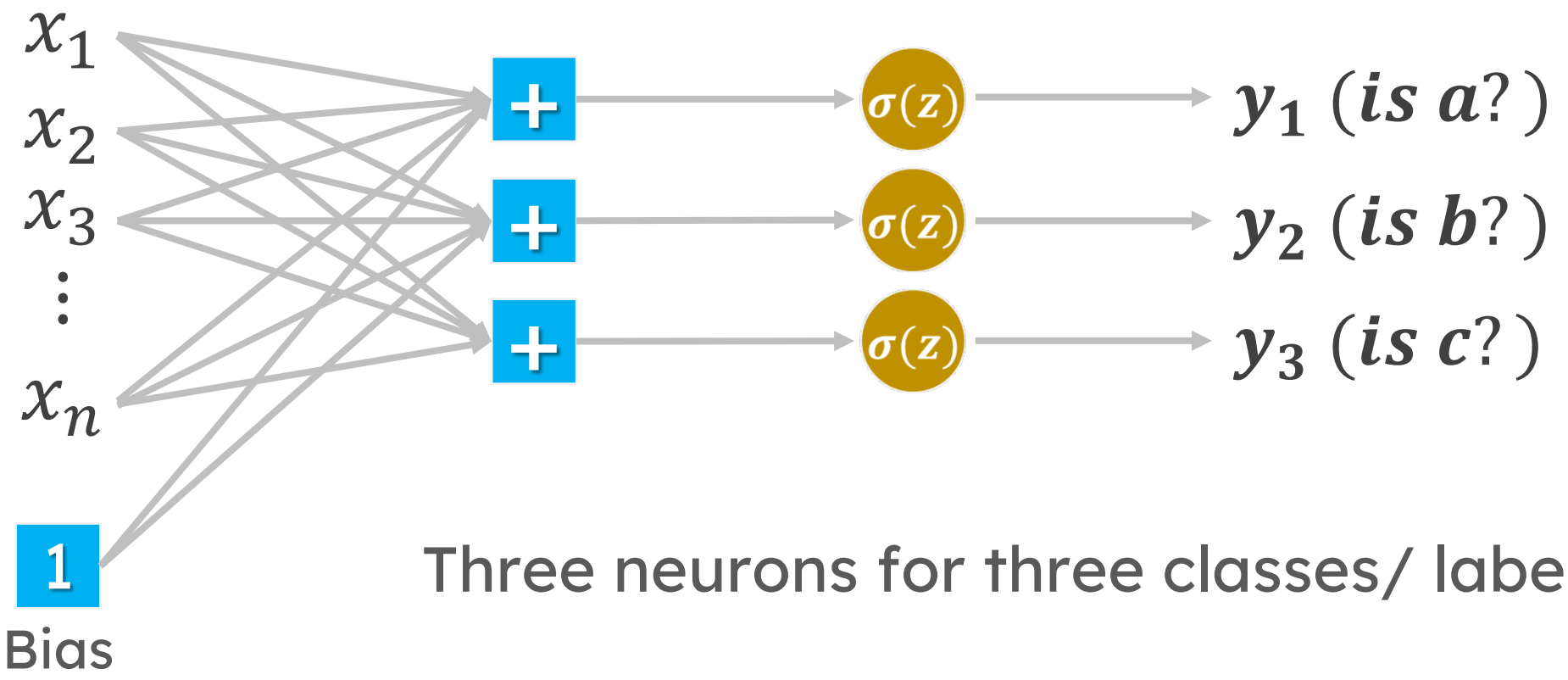


Single Neuron



$$y = h_{w,b}(x)$$
$$= \sigma(w^T x + b)$$

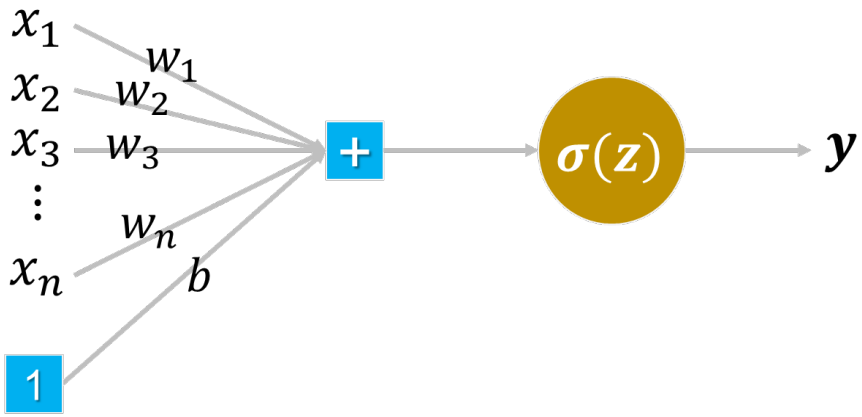
A Layer of Neurons :: Perceptron



Choose the max one

Three neurons for three classes/ labels

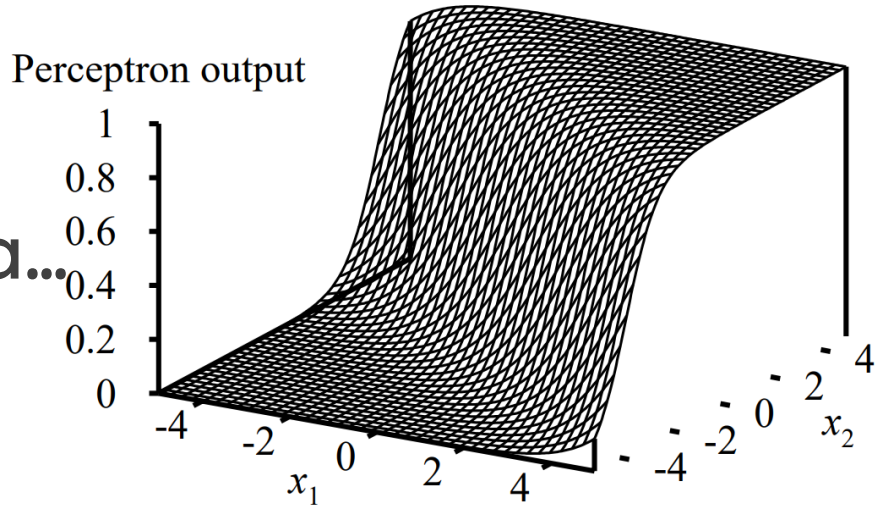
A Layer of Neurons :: Perceptron



$$\begin{cases} true & y \geq \theta \\ false & y < \theta \end{cases}$$

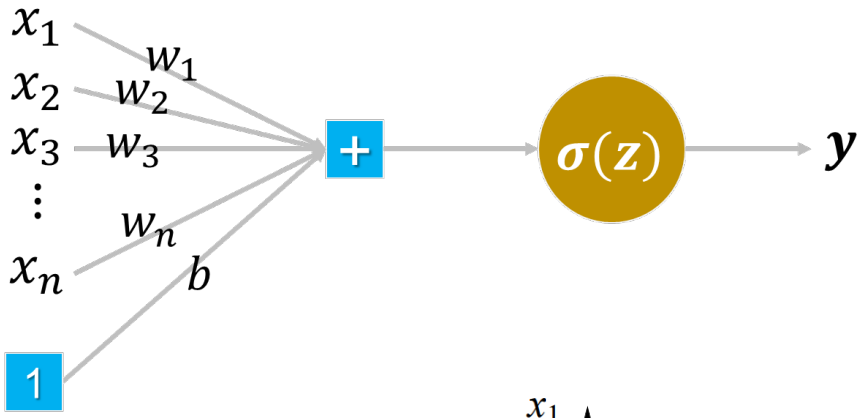
θ *is threshold*

$y = \sigma(w_1 x_1 + w_2 x_2 + b)$ is a linear formula...

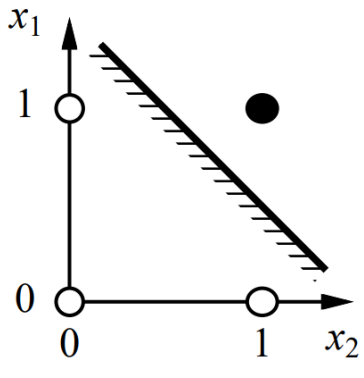


Source: https://aima.cs.berkeley.edu/slides-pdf/chapter20b.pdf

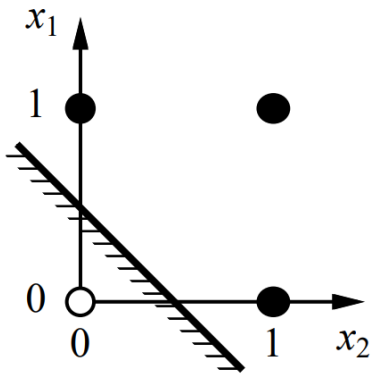
A Layer of Neurons :: Perceptron



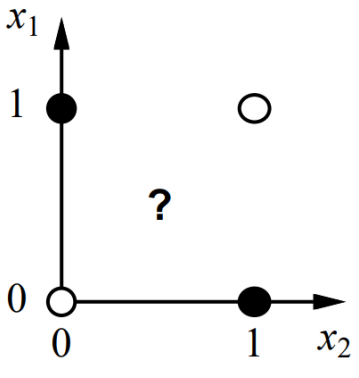
$$\begin{cases} true & y \geq \theta \\ false & y < \theta \end{cases} \qquad \theta \text{ is threshold}$$



(a) x_1 **and** x_2



(b) x_1 **or** x_2

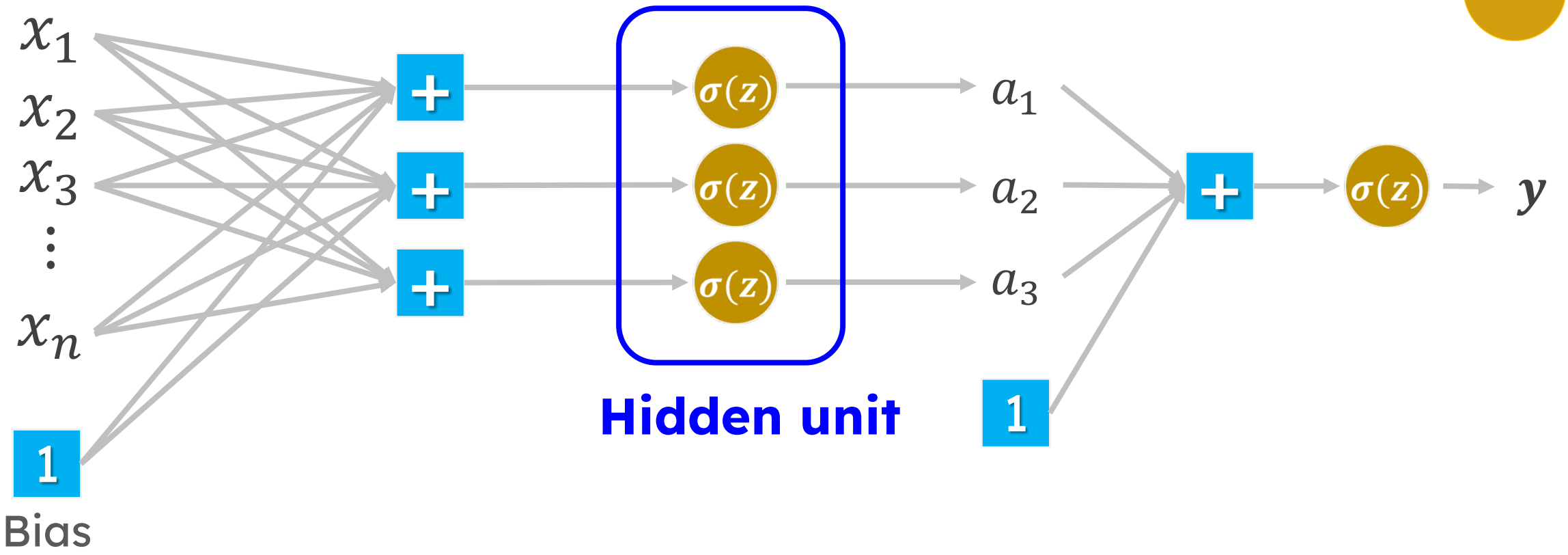


(c) x_1 **xor** x_2

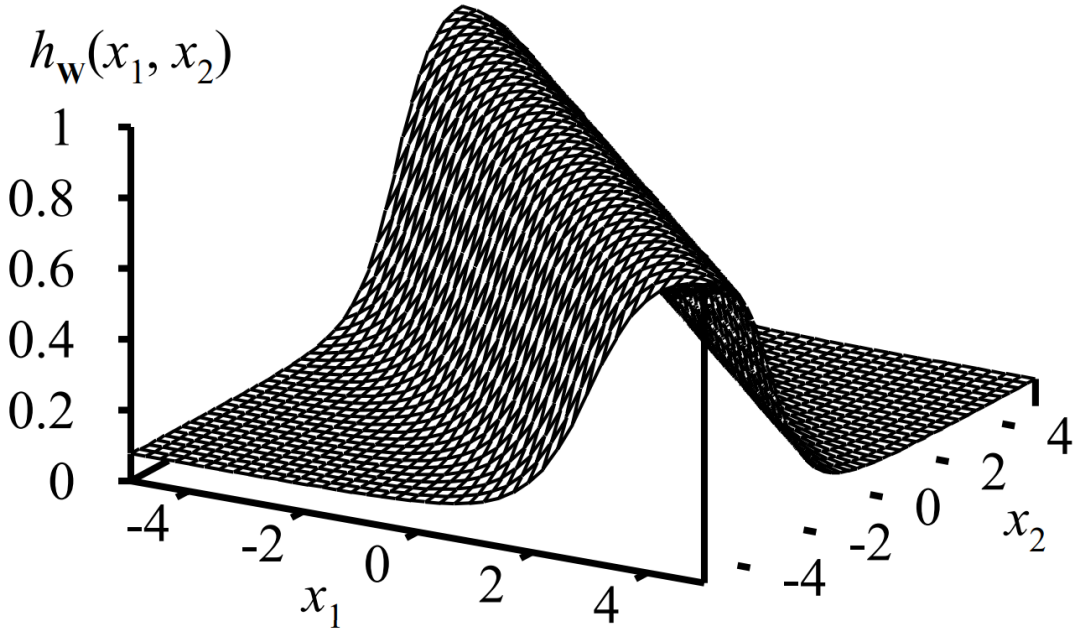
Minsky & Papert (1969) pricked the neural network balloon

Source: https://aima.cs.berkeley.edu/slides-pdf/chapter20b.pdf

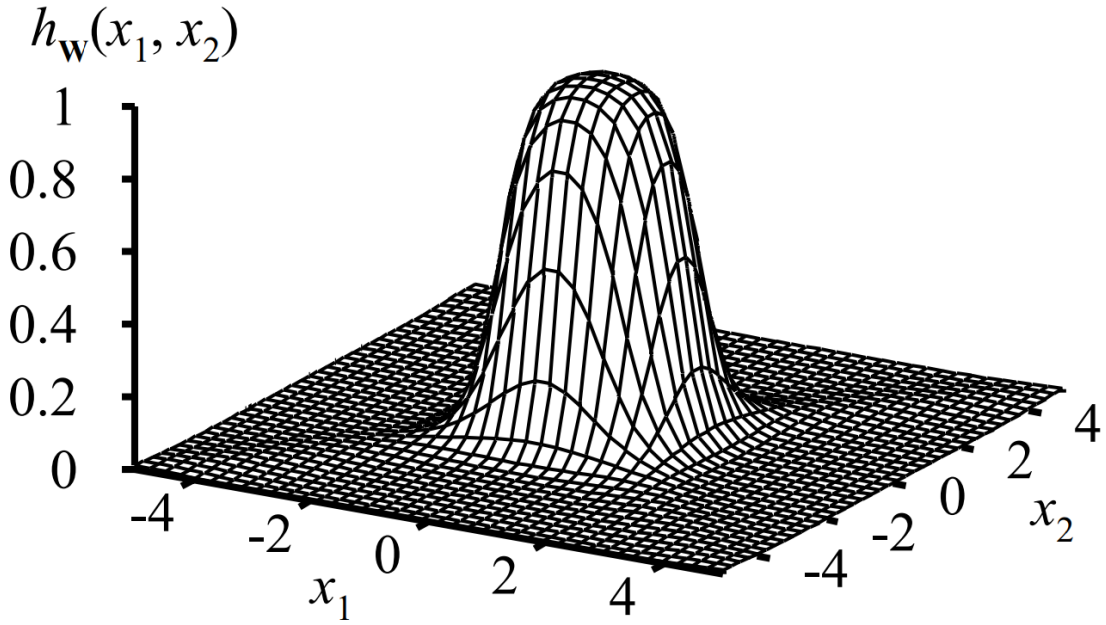
A Layer of Neurons :: Perceptron



A Layer of Neurons :: Perceptron



Combine two opposite-facing threshold functions to make a ridge

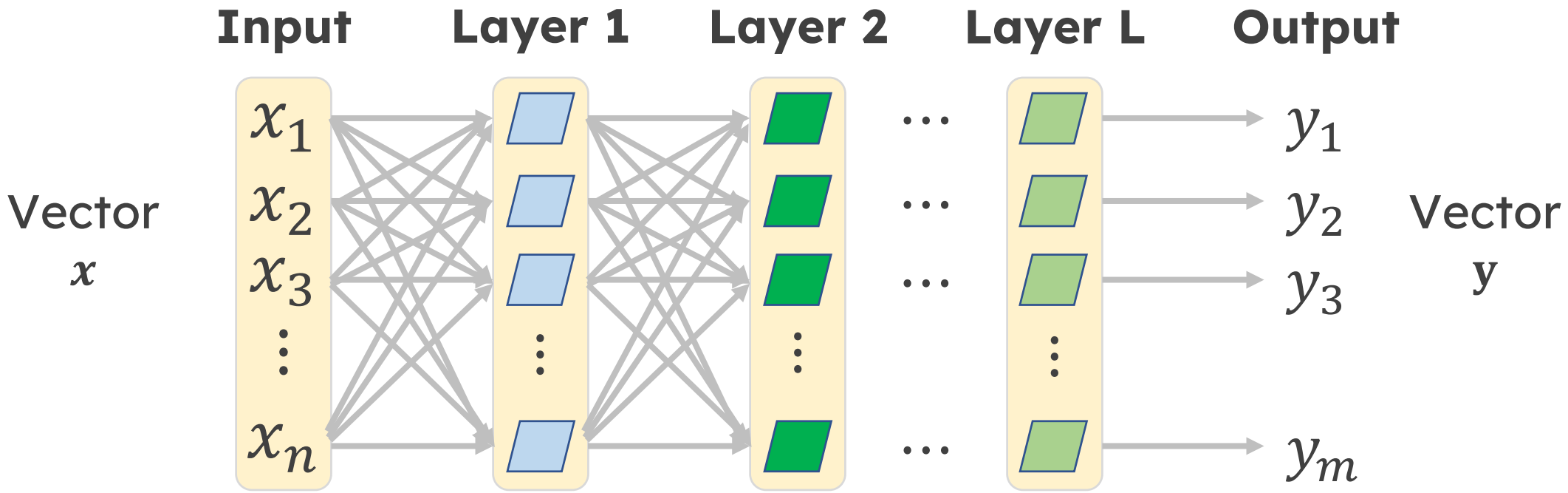


Combine two perpendicular ridges to make a bump

Source: https://aima.cs.berkeley.edu/slides-pdf/chapter20b.pdf

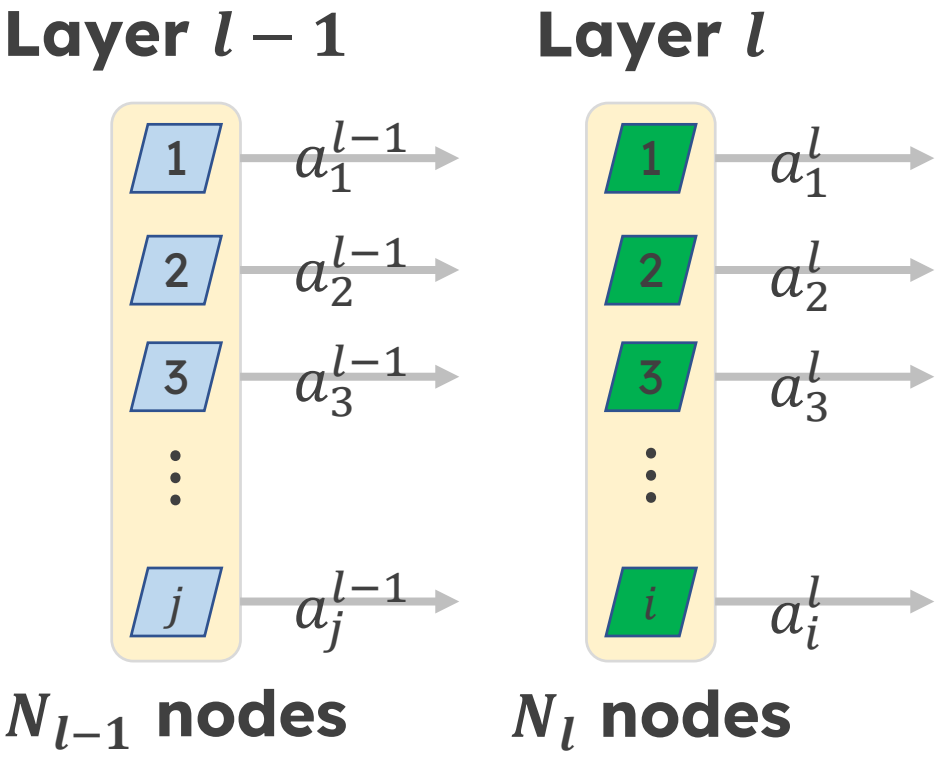
Fully Connected Feedforward Network

Deep Neural Network (DNN)



DNN: multiple hidden layers

Hidden Layer :: Layer



A Neuron Output

a_1^l → Layer l

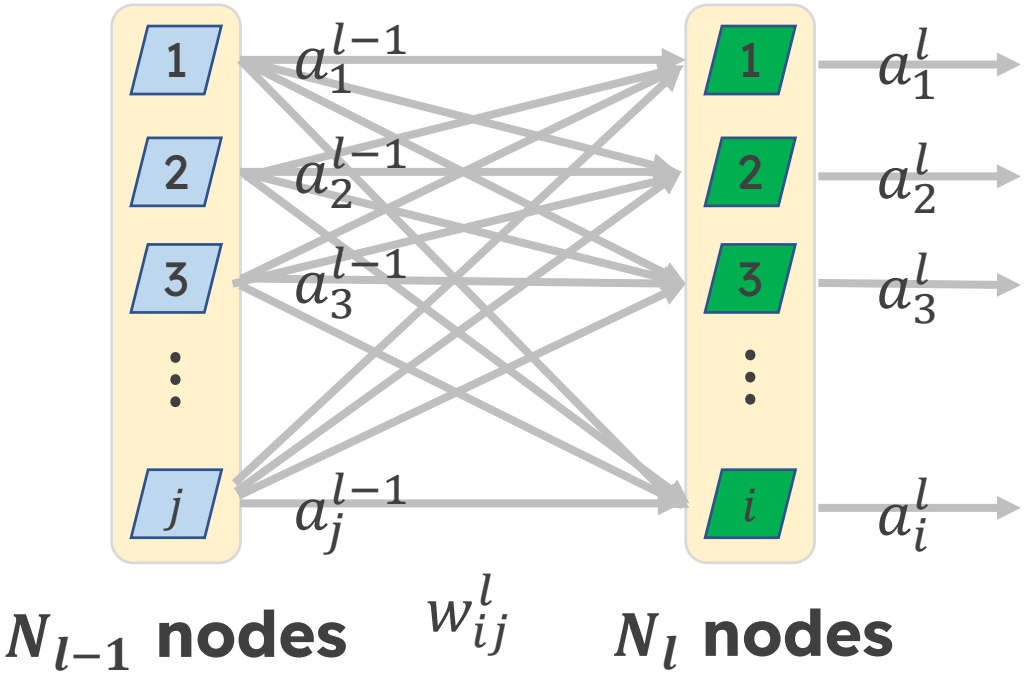
a_1^l → neuron i

$$a^l = \begin{bmatrix} \vdots \\ a_i^l \\ \vdots \end{bmatrix}$$

One layer ➔ One Vector

Hidden Layer :: Weights

A Weight between two neurons/nodes



w_{ij}^l

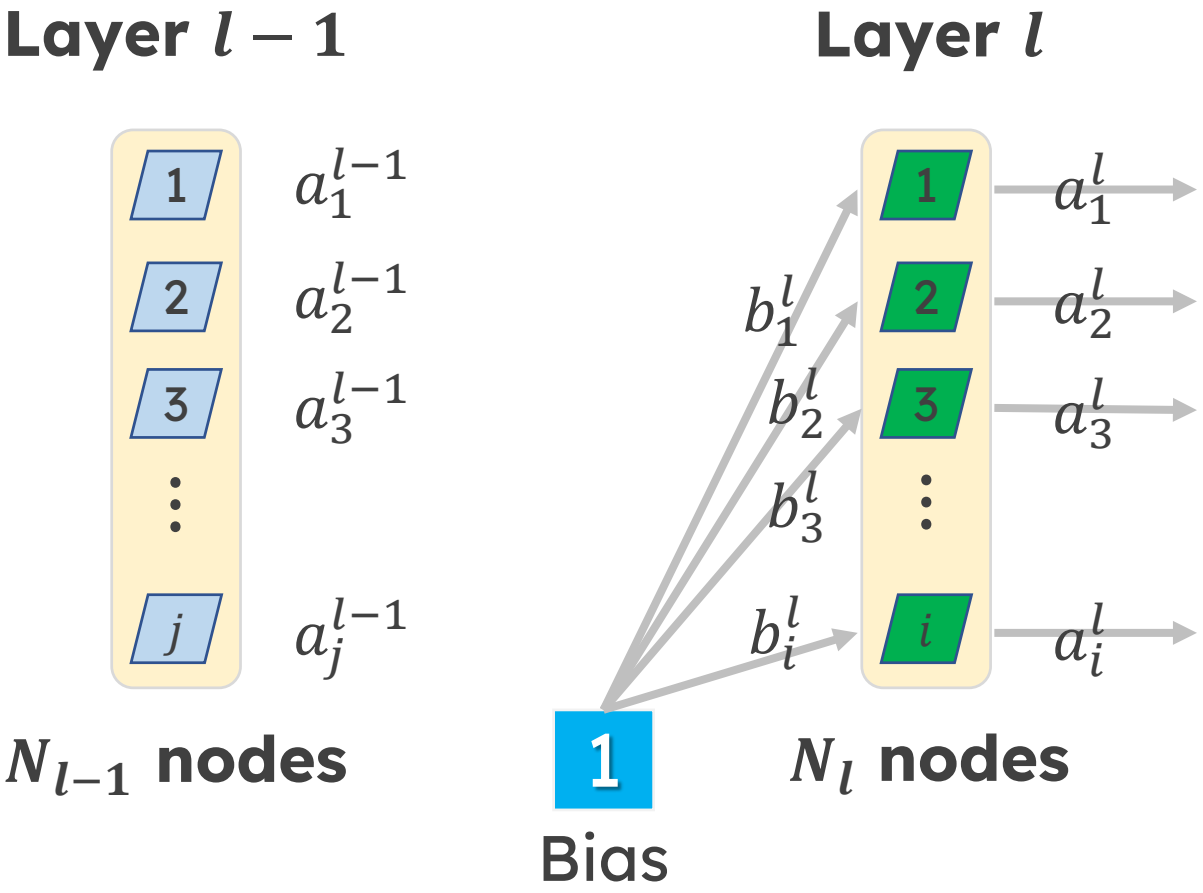
- Layer $l-1$ to Layer l
- From neuron j (layer $l-1$) to neuron i (layer l)

$$w^l = \begin{bmatrix} w_{11}^l & w_{12}^l & \cdots \\ w_{21}^l & w_{22}^l & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

N_{l-1} (columns), N_l (rows)

Weights between layers ➔ One Matrix

Hidden Layer :: Bias

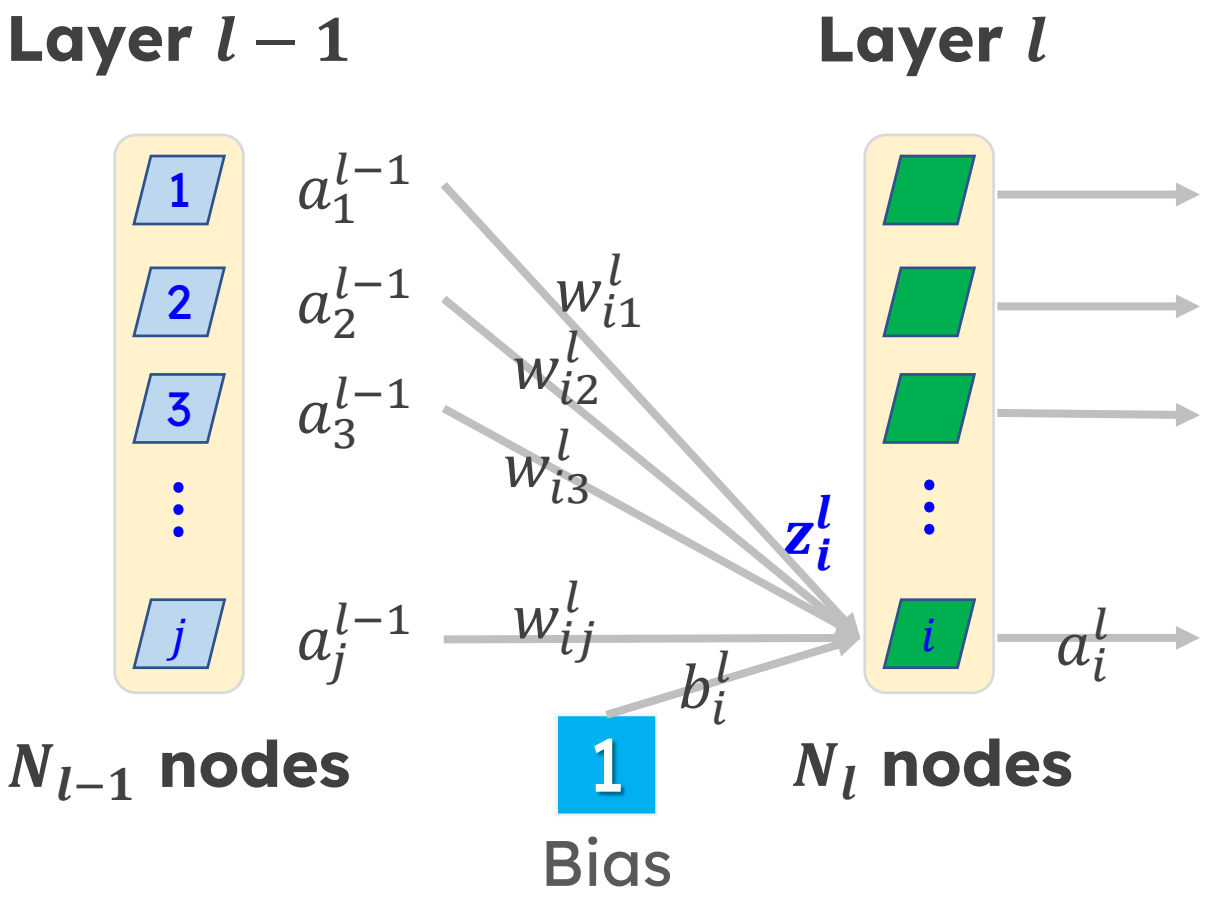


b_i^l Bias for neuron i at layer

$$b^l = \begin{bmatrix} \vdots \\ b_i^l \\ \vdots \end{bmatrix}$$

Bias of each layer ➔ One Vector

Hidden Layer :: Activation Function



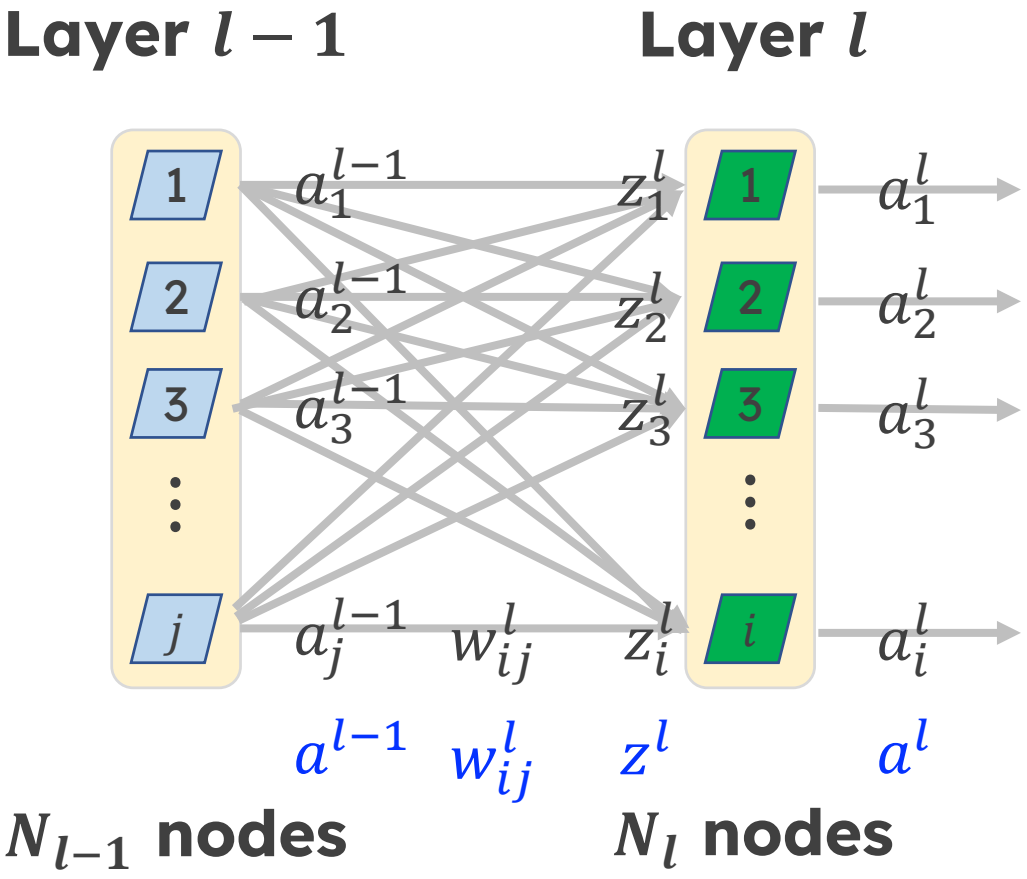
$$z_i^l = w_{i1}^l a_1^{l-1} + w_{i2}^l a_2^{l-1} + \cdots + b_i^l$$

$$z_i^l = \sum_{j=1}^{N_{l-1}} w_{ij}^l a_j^{l-1} + b_i^l$$

$$z^l = \begin{bmatrix} \vdots \\ z_i^l \\ \vdots \end{bmatrix}$$

Activation function at each layer ➔ One Vector

Hidden Layer :: Overall



$$z_1^l = w_{11}^l a_1^{l-1} + w_{12}^l a_2^{l-1} + \cdots + b_1^l$$

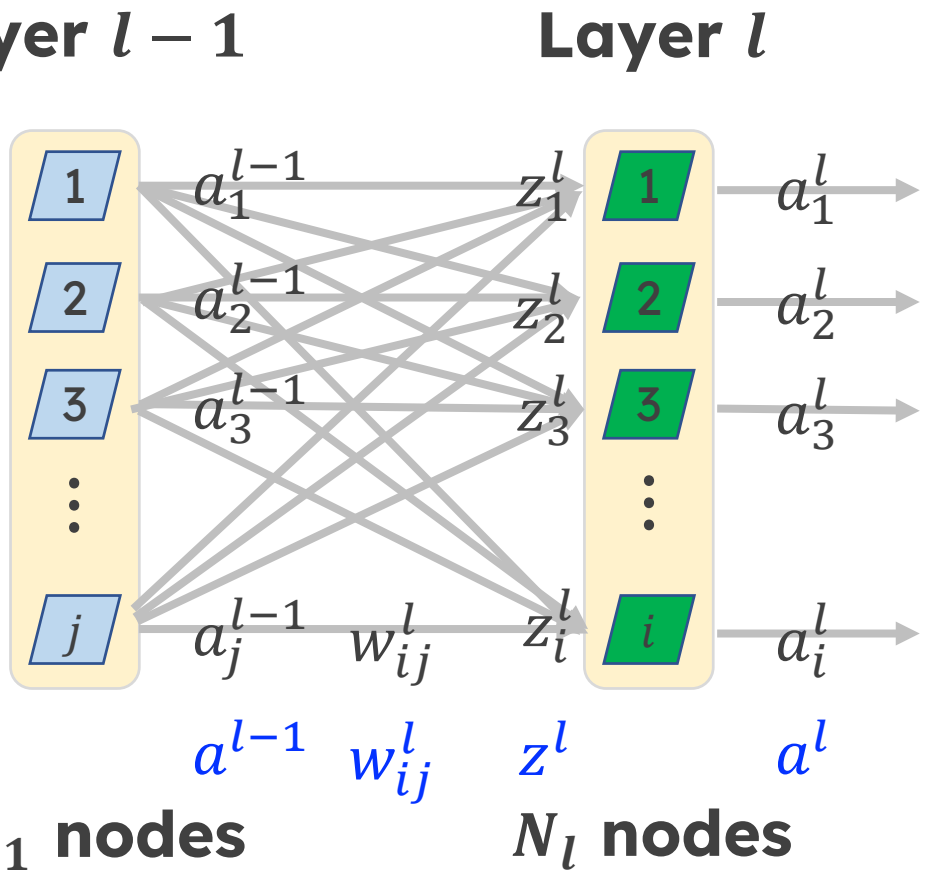
$$\vdots$$

$$z_i^l = w_{i1}^l a_1^{l-1} + w_{i2}^l a_2^{l-1} + \cdots + b_i^l$$

$$\begin{bmatrix} \vdots \\ z_i^l \\ \vdots \end{bmatrix} = \begin{bmatrix} w_{11}^l & w_{12}^l & \cdots \\ w_{21}^l & w_{22}^l & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} \vdots \\ a_i^{l-1} \\ \vdots \end{bmatrix} + \begin{bmatrix} \vdots \\ b_i^l \\ \vdots \end{bmatrix}$$

$$z^l = W^l a^{l-1} + b^l$$

Hidden Layer :: Overall

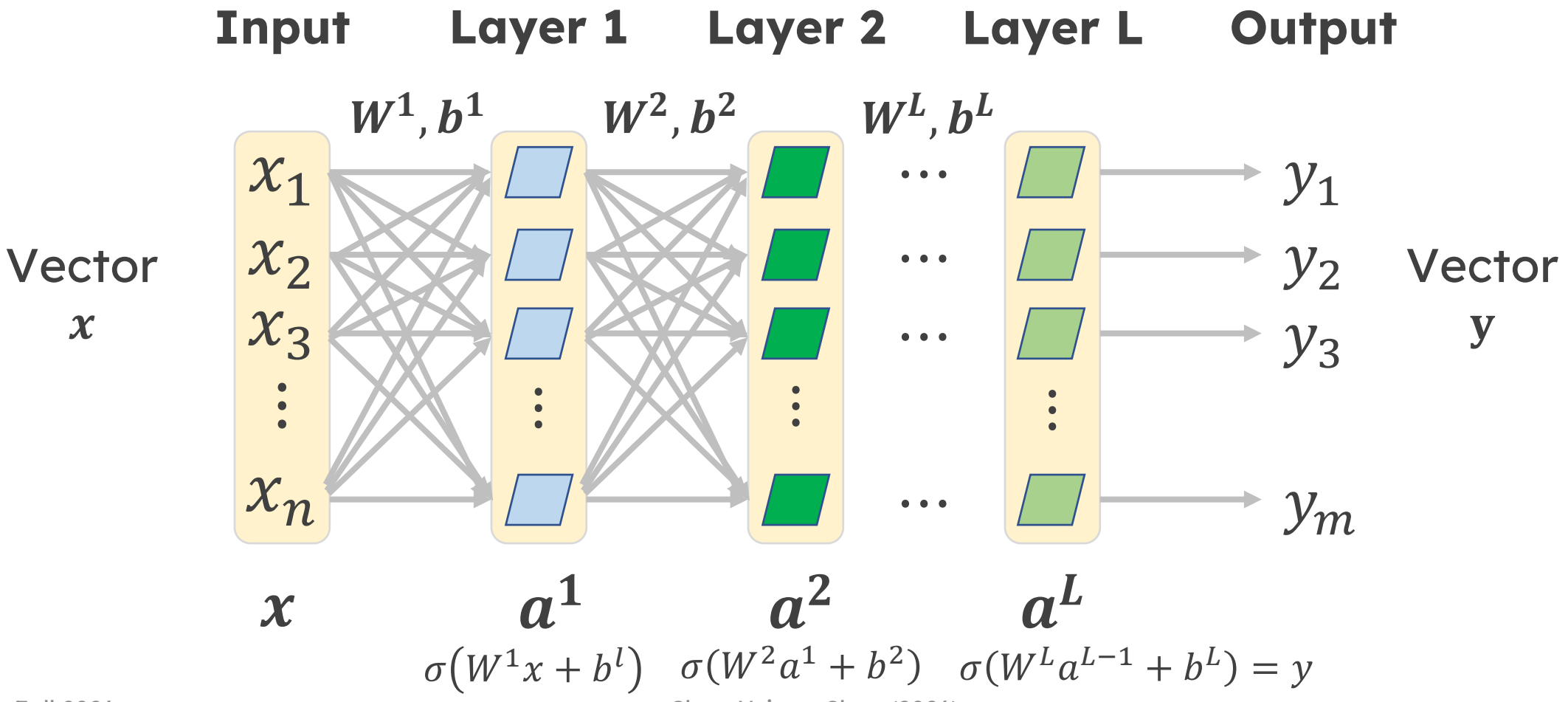


$$a_i^l = \sigma\left(z_i^l\right)$$

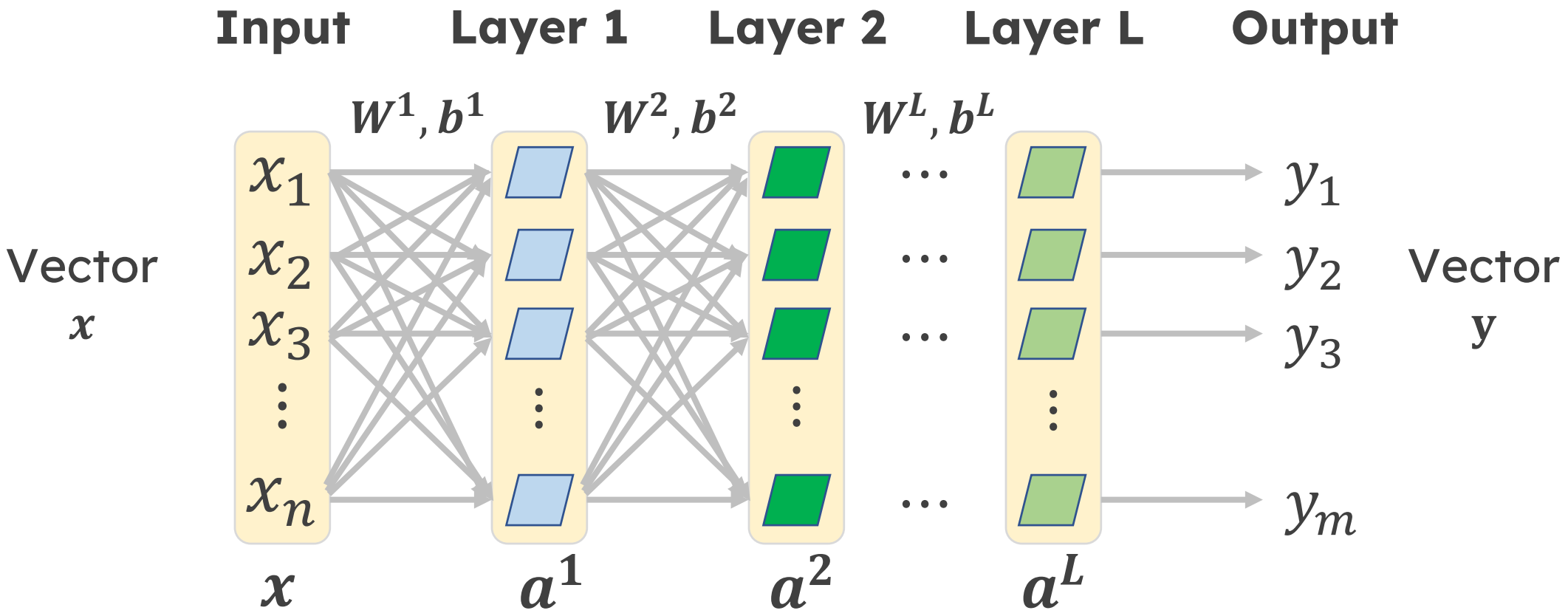
$$\begin{bmatrix} a_1^l \\ a_2^l \\ \vdots \\ a_i^l \end{bmatrix} = \begin{bmatrix} \sigma\left(z_1^l\right) \\ \sigma\left(z_2^l\right) \\ \vdots \\ \sigma\left(z_i^l\right) \end{bmatrix}$$

$$a^l = \sigma\left(z^l\right)$$

Hidden Layer :: Overall

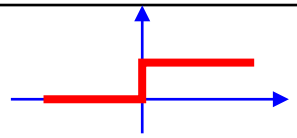
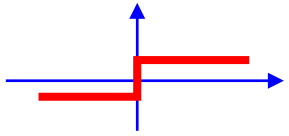
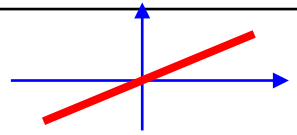
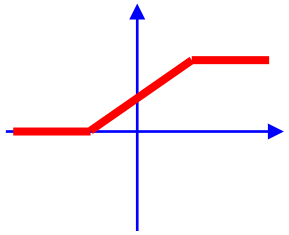
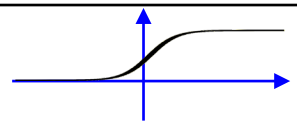
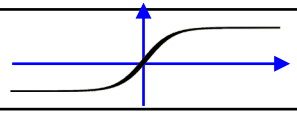


Hidden Layer :: Overall



$$y = f(x) = \sigma(W^L \dots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \dots + b^L)$$

Activation Function

Activation Function	Equation	Example	1D Graph
Unit step (Heatviside)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise Linear	$\phi(z) = \begin{cases} 1 & z \geq \frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} < z < \frac{1}{2} \\ 0 & z \leq -\frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multi-layer NN	
Hyperbolic Tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer NN	

Activation Function

Sigmoid

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

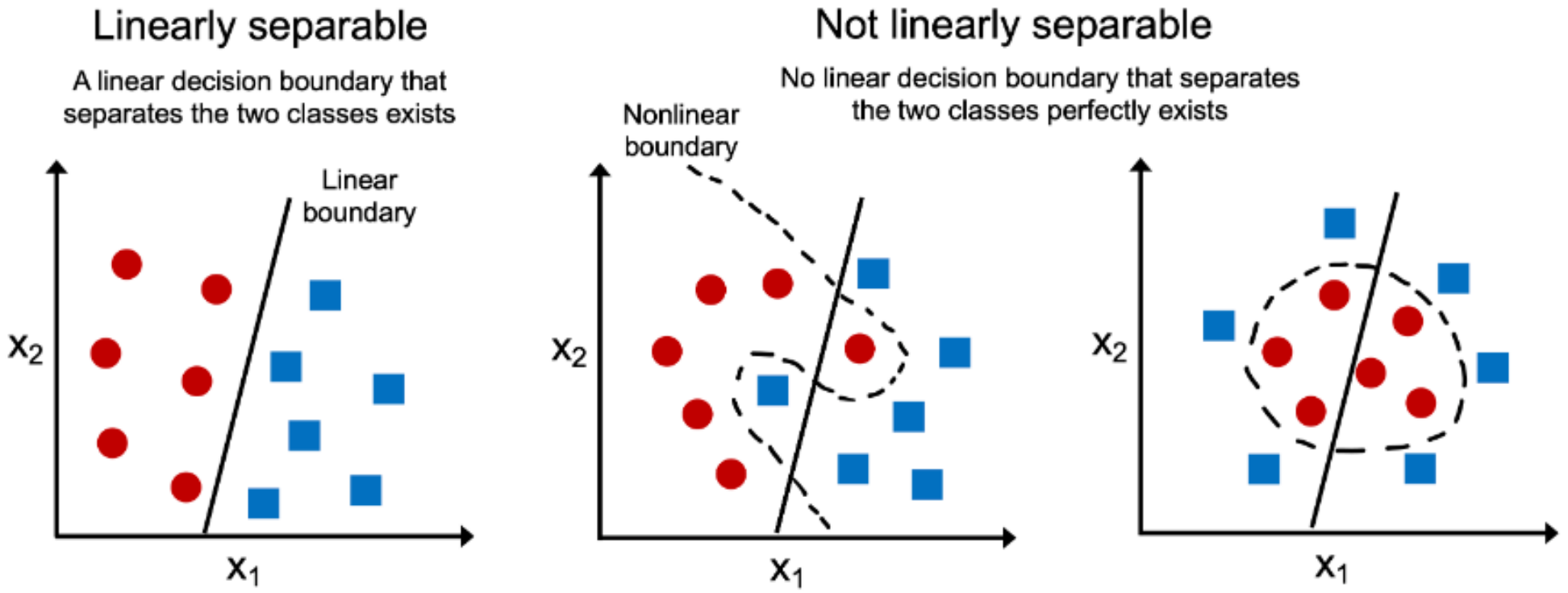
Tanh

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Rectified Linear Unit (ReLU)

$$ReLU(x) = \max(x, 0)$$

Linearity and Non-linearity



Source: https://vitalflux.com/how-know-data-linear-non-linear/

Linearity and Non-linearity

With linearity, the deep neural network is the same as linear transform.

$$W_1(W_2 \cdot x) = (W_1 W_2)x = Wx$$

With non-linearity, the deep neural network with multiple layers could have a more complex function.

Model Parameters

$$y = f(x) = \sigma(W^L \dots \sigma(W^2\sigma(W^1 x + b^1) + b^2) \dots + b^L)$$

Function set

Different parameters W and b → different functions

Formal definition

$$f(x;\theta) \Rightarrow model\ parameter\ set$$
$$\theta = \{W^1, b^1, W^2, b^2, \dots, W^L, b^L\}$$

Loss Function

Define a function to measure the quality of parameters set θ

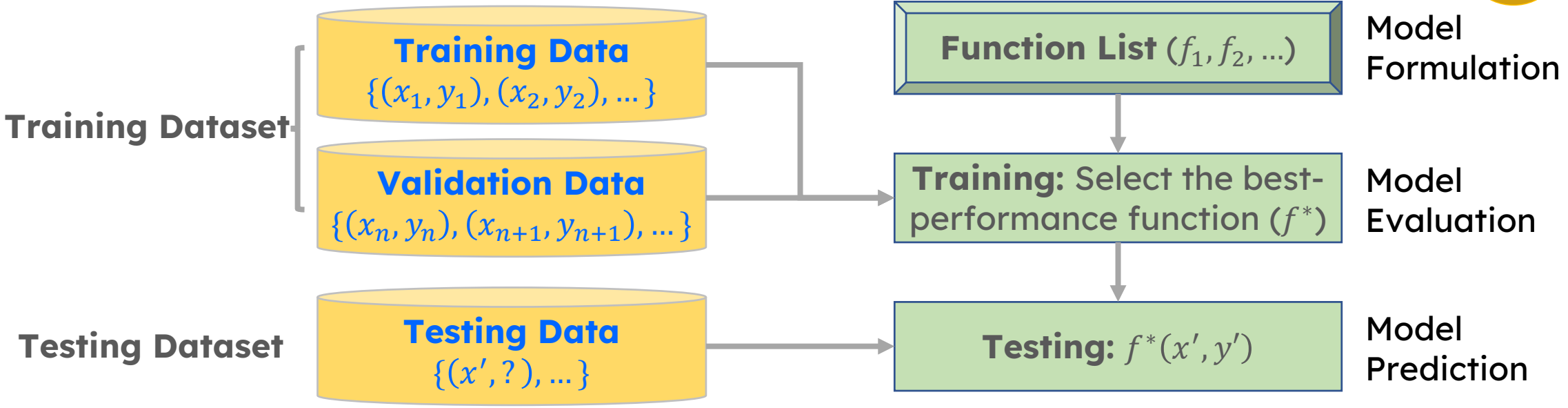
Evaluating by a loss/cost/error function $C(\theta)$ ➔ how bad

$$\theta^* = \arg\min_{\theta} C(\theta)$$

Evaluating by an objective/reward function $O(\theta)$ ➔ how good

$$\theta^* = \arg\max_{\theta} O(\theta)$$

Loss Function



Best function: $f(x;\theta) \sim \hat{y}$ ➔ $\|\hat{y} - f(x;\theta)\| \approx 0$

Define a loss function: $C(\theta) = \sum_k \|\hat{y}_k - f(x_k;\theta)\|$

Loss Function

Table 1 List of losses analyzed in this paper. y is true label as one-hot encoding, $\hat{y}$ is true label as $+1/-1$ encoding, o is the output of the last layer of the network, $\cdot\ (j)$ denotes $j-th$ dimension of a given vector, and $\sigma(\cdot)$ denotes probability estimate.

Symbol	Name	Equation
$\mathcal{L}_1$	L_1 loss	$\|y-o\|_1$
$\mathcal{L}_2$	L_2 loss	$\|y-o\|_2^2$
$\mathcal{L}_1 \circ \sigma$	Expectation loss	$\|y-\sigma(o)\|_1$
$\mathcal{L}_2 \circ \sigma$	Regularized expectation loss	$\|y-\sigma(o)\|_2^2$
$\mathcal{L}_\infty \circ \sigma$	Chebyshev loss	$max_j\|\sigma(o)^j - y^j\|$

Source: Janocha, K., & Czarnecki, W. M. (2017). On loss functions for deep neural networks in classification. *arXiv preprint arXiv:1702.05659*.

Loss Function

Table 1 List of losses analyzed in this paper. y is true label as one-hot encoding, $\hat{y}$ is true label as $+1/-1$ encoding, o is the output of the last layer of the network, $\cdot\ (j)$ denotes $j-th$ dimension of a given vector, and $\sigma(\cdot)$ denotes probability estimate.

Symbol	Name	Equation
$Hinge$	Hinge (margin) los	$\sum_j \max\left(0, \frac{1}{2} - \hat{y}^j o^j\right)$
$Hinge^2$	Squared hinge (margin) loss	$\sum_j \max\left(0, \frac{1}{2} - \hat{y}^j o^j\right)^2$
$Hinge^3$	Cubed hinge (margin) loss	$\sum_j \max\left(0, \frac{1}{2} - \hat{y}^j o^j\right)^3$

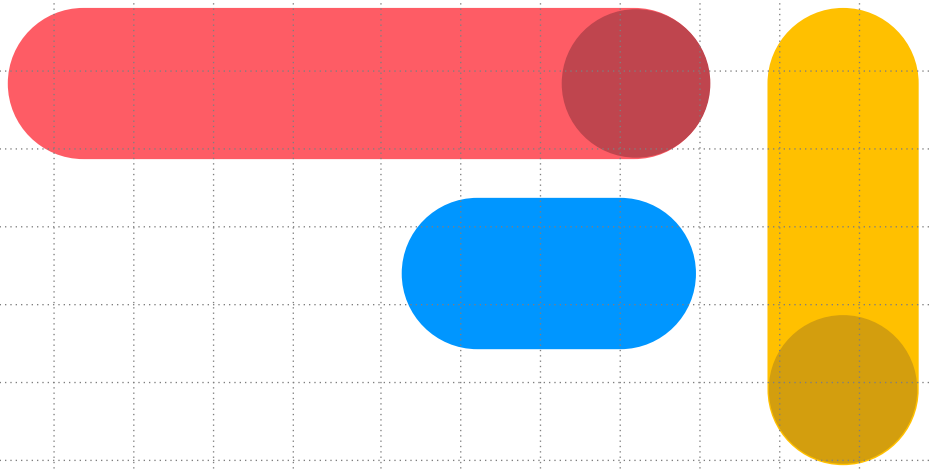
Loss Function

Table 1 List of losses analyzed in this paper. y is true label as one-hot encoding, $\hat{y}$ is true label as $+1/-1$ encoding, o is the output of the last layer of the network, $\cdot\ (j)$ denotes $j-th$ dimension of a given vector, and $\sigma(\cdot)$ denotes probability estimate.

Symbol	Name	Equation
log	Log (cross entropy) loss	$-\sum_j y^j log\sigma(o)^j$
$\log^2$	Squared log loss	$-\sum_j \left[y^j log\sigma(o)^j\right]^2$
tan	Tanimoto loss	$\frac{-\sum_j \sigma(o)^j y^j}{\|\sigma(o)\|_2^2 + \|y\|_2^2 - \sum_j \sigma(o)^j y^j}$

References

1) 臺大電機系李宏毅教授講義
2) 臺大資工系陳縕儂教授講義
3) NVIDIA Deep Learning Tutorial
4) https://aima.cs.berkeley.edu/slides-pdf/chapter20b.pdf
5) Janocha and Czarnecki (2017) On Loss Functions for Deep Neural Networks in Classification. *Computer Science ArXiv*. abs/1702.05659.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: https://toodou.github.io/